RÉSUMÉ : API GRAPHQL

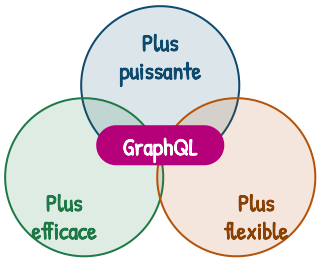
Partie 1/2 — Pourquoi GraphQL ? GraphQL vs REST

Badia Bouhdid (github.com/badi3a)

Architecture Orientée Services
Chapitre 5 · 2026-2027

1 LE BESOIN

- **Mobiles** : charger les données efficacement, en transférant le moins possible sur le réseau.
- **Front-ends variés** (web, mobile, frameworks…) : chacun a ses besoins → c'est au **client** de fixer la structure de la réponse.



2 QU'EST-CE QUE GRAPHQL ?

GraphQL : **langage de requête** pour API et **environnement d'exécution**. Créé chez Facebook en 2012, open source depuis 2015.

- Un **seul point d'accès** (souvent `/graphql` sur HTTP).
- Les données sont décrites par un **schéma typé** : un graphe de types.
- Le client demande **exactement** les champs dont il a besoin.

API REST : plusieurs URI

```
Client → /movies
Client → /movies/1
Client → /directors
```

API GraphQL : un seul endpoint

```
Client → /graphql
```

movies, directors… dans une seule requête

3 PROBLÈME 1 : LA SUR-EXTRACTION (OVER-FETCHING)

On télécharge **plus de données que nécessaire**. Besoin : le titre des 2 derniers films.

REST

```
GET /movies
```

× renvoie **tous les films** avec **tous leurs champs**, même ceux qui ne servent pas au traitement.

GraphQL

```
{ movies(last: 2) { title } }
```

```
{ "data": { "movies": [
  { "title": "Titanic" },
  { "title": "X-Man" } ] } }
```

✓ seulement les champs demandés

4 PROBLÈME 2 : LA SOUS-EXTRACTION (UNDER-FETCHING)

Une requête ne suffit pas : il faut **plusieurs appels**. Besoin : le film 1, ses réalisateurs et leurs 3 derniers films.

REST : 3 requêtes

- `GET /movies/1` → infos du film
- `GET /directors?idM=1` → réalisateurs
- `GET /movies?count=3` → 3 derniers films

× plusieurs allers-retours réseau, un par URI

GraphQL : 1 requête imbriquée

✓ toutes les données en un seul appel, dans la forme demandée

```
{
  movies(id: 1) {
    title
    directors {
      name
      movies(last: 3) {
        date
      }
    }
  }
}
```

5 REST VS GRAPHQL

Critère	REST	GraphQL
Points d'accès	une URI par ressource	un seul endpoint
Structure de la réponse	fixée par le serveur	choisie par le client
Opérations	GET (lire), POST (créer), PUT (modifier), DELETE	**Query** (lire), **Mutation** (créer, modifier, supprimer), **Subscription** (temps réel)
Méthode HTTP	dépend de l'opération	surtout POST (GET possible pour une query)
Format des données	JSON, XML, texte, HTML…	JSON (le plus courant)
Transport	HTTP	indépendant du transport ; HTTP en pratique
Contrat	facultatif (OpenAPI)	schéma typé obligatoire
Cache HTTP	natif (GET sur une URI)	plus complexe (un seul endpoint)

→ Aucun n'est « plus rapide » par nature : tout dépend des resolvers, du cache et du réseau. GraphQL est une alternative, pas un remplaçant universel.

RÉSUMÉ : API GRAPHQL

Partie 2/2 — Schéma, requêtes, mutations et resolvers

Badia Bouhdid (github.com/badi3a)

Architecture Orientée Services
Chapitre 5 · 2026-2027

6 LE SCHÉMA ET SES OPÉRATIONS

Schéma : élément central d'une API GraphQL, écrit en **SDL** (Schema Definition Language). C'est le **contrat** entre client et serveur.

On y définit les **types d'objets** (champs et types), les **opérations** disponibles et des règles (ex. ! = non nul).

- **Query** — lire des données (≈ GET en REST)
- **Mutation** — créer, modifier, supprimer (≈ POST, PUT, DELETE)
- **Subscription** — recevoir des mises à jour en **temps réel** (souvent via WebSocket)

7 TYPES ET OBJETS

```
type Book {
  id: ID!
  title: String
  pageCount: Int
  authors: [Author]
}
```

→

```
type Author {
  firstName: String
  lastName: String
}
```

! = **non nul** : le champ est toujours renvoyé ; sur un argument, il est **obligatoire**.

[Author] = liste → relation entre objets.

Scalaires : `Int, Float, String, Boolean, ID.`

8 QUERY : LIRE DES DONNÉES

Définition (schéma)

```
type Query {
  book(id: ID!): Book
  books: [Book]
  allAuthors(count: Int):
    [Author]
}
```

Requête du client

```
{
  books {
    title
    authors {
      firstName
    }
  }
}
```

Réponse (JSON)

```
{ "data": {
  "books": [
   { "title": "L'œil et le mot",
     "authors": [
     { "firstName": "Jean-François" }
     ] },
   …
  ] } }
```

→ La réponse a **exactement la forme de la requête**, dans un objet **data** : on ne reçoit que les champs demandés.

9 MUTATION : CRÉER, MODIFIER, SUPPRIMER

Définition (schéma)

```
type Mutation {
  createBook(id: ID!,
    title: String,
    pageCount: Int): Book
  deleteBook(id: ID!): Book
  addAuthor(
    firstName: String!,
    lastName: String!): Author
}
```

Requête du client

```
mutation {
 createBook(id: "123",
  title: "L'œil et le mot",
  pageCount: 240) {
   id
   title
 }
}
```

Réponse (JSON)

```
{ "data": {
  "createBook": {
    "id": "123",
    "title":
      "L'œil et le mot"
  } } }
```

Supprimer : `mutation { deleteBook(id: "123") { id title } }` — une mutation renvoie, elle aussi, les champs demandés.

10 LES RESOLVERS : LA LIAISON AVEC LE MÉTIER

Resolver : fonction associée à un champ du schéma, qui va chercher sa donnée dans **n'importe quelle source** (base de données, fichiers, autre API…). Le schéma dit *quoi* ; les resolvers disent *comment*.



POINTS À RETENIR

Un **endpoint**, un **schéma typé** ; le client choisit ses champs → fin de la sur- et de la sous-extraction.

Query = lire · **Mutation** = écrire · **Subscription** = temps réel ; ! = non nul.

Chaque champ est calculé par un **resolver**, qui va chercher la donnée dans sa source.